

# Password Spraying

Every guide so far in this series has assumed an **offline** attack — you already have a dumped hash and you're cracking it locally, unconstrained by rate limits, lockouts, or anyone watching. Password spraying flips that model entirely: it's an **online** attack against a live authentication endpoint, and the entire technique exists specifically to route *around* the one defense your JtR/Hashcat/Rainbow Tables guides kept listing almost as an afterthought — "rate-limit/lock out online authentication attempts, irrelevant to offline cracking." This guide is where that defense actually gets tested directly. Worth locking in the distinction immediately: **Part 1 covers the core mechanics** — why spraying works, and specifically why it evades lockout policies that would stop a naive brute-force attempt instantly — **Part 2 covers practical execution, tooling, and target selection** — how this actually gets run against real environments (Active Directory, cloud identity providers, web apps) without tripping the defenses it's designed around.

---

## 1. Why Spraying Inverts the Normal Brute-Force Model — The Foundation

Traditional online brute-forcing picks **one account** and tries **many passwords** against it. Nearly every authentication system defends against exactly this pattern with an account lockout policy:

```
Traditional brute-force:
target_user + guess_1, guess_2, guess_3, guess_4, guess_5, ...
→ after N failed attempts (commonly 3-10), the ACCOUNT LOCKS,
   attack stops dead, and a lockout event is a loud, easily
   alerted-on signal for defenders
```

```
Password spraying INVERTS this entirely:
guess_1 + user_1, user_2, user_3, ... user_1000
guess_2 + user_1, user_2, user_3, ... user_1000
(wait out the lockout policy's reset window)
guess_3 + user_1, user_2, user_3, ... user_1000
```

**The single fact that makes this work — lockout policies count failed attempts PER ACCOUNT, not per password or per source, so trying one password against a thousand different accounts triggers at most ONE failed attempt**

**per account, staying safely under any reasonable lockout threshold, while trying a thousand passwords against one account triggers a thousand failed attempts against that single account and locks it almost immediately.**

Spraying doesn't defeat lockout policies through cleverness — it simply never generates the per-account failure count that would trigger one in the first place.

Typical lockout policy: 5 failed attempts / 30 minute observation window / account locks for 30 minutes

Spraying against this: try 1 password across ALL target accounts, wait 30+ minutes (or longer, with margin), try the NEXT password across all accounts again - each account only ever accumulates 1 failed attempt per "round," permanently staying under the 5-attempt threshold

## 2. Why It Works So Well Despite Trying "Only" a Handful of Passwords

Spraying deliberately sacrifices Section 1's traditional brute-force depth (many guesses per account) for **breadth** (one guess across a huge account population) — and this tradeoff pays off precisely because of a statistical reality your Wordlists guide's Section 1 already established: a small number of extremely common passwords account for a disproportionate share of real-world password choices across ANY sufficiently large population.

Against a population of 1,000 real users, trying just 5-10 of the MOST COMMON passwords (Summer2026!, Password123!, CompanyName123!, Welcome1, etc.) will statistically succeed against SOME percentage of that population, almost every single time - the "weakest link" principle means you only need ONE of a thousand users to have chosen a common password for the spray to succeed

This is the direct real-world payoff of your Wordlists guide's "probable wordlists, frequency-ranked" concept (Section 4 there) — spraying is specifically optimized around trying the highest-probability candidates first, since the attack's entire viability depends on succeeding within a very small number of attempts per account.

---

## PART 1: Core Mechanics & Target Selection

### 3. Building the Target Username List

Before spraying anything, you need the account population itself — this is a reconnaissance step with no direct equivalent in your earlier offline-cracking guides, since there you already had a dumped set of accounts by definition:

Common username enumeration sources:

- OSINT: LinkedIn employee names, company "about us"/staff pages, press releases, conference speaker lists (per your Wordlists guide's Section 9 OSINT-compilation approach, same technique, different target - usernames instead of password base terms)
- Email format inference: observing ONE known valid address (firstname.lastname@company.com) and inferring the pattern applies organization-wide
- Username enumeration vulnerabilities: login pages, password reset flows, or registration forms that respond DIFFERENTLY for a valid vs invalid username (a distinct vulnerability class worth flagging - directly enables this reconnaissance step)
- Breach data: previously breached credential dumps from the SAME organization often retain valid usernames/email format even when the associated passwords have since been rotated

## 4. Calculating Safe Timing — Working Within the Lockout Window

The core operational parameter of any spray is the delay between rounds, derived directly from the target's lockout policy:

Lockout threshold: N failed attempts

Lockout observation window: W minutes (the window resets counting if enough time passes without a NEW failed attempt)

Safe round interval = W + safety margin (commonly W + 5-15 minutes, to account for clock drift/policy misunderstanding)

Example: 5-attempt threshold, 30-minute window

→ wait AT LEAST 30-45 minutes between spray rounds, trying at most 1 (or a small handful, depending on margin preference) password per account, per round

Discovering the target's actual lockout policy in advance (rather than guessing) matters enormously here — Active Directory environments commonly expose this via `net accounts /domain` (once any foothold exists) or it can sometimes be inferred by deliberately testing against a single disposable/test account before the real spray begins.

## 5. Choosing the Password List — Small, High-Probability, Never Generic Bulk

This is the single sharpest departure from your JtR/Hashcat/Wordlists guides' approach — spraying explicitly does NOT want a 14-million-entry rockyou.txt-

style list, because every single password in the list requires a full lockout-window wait cycle across the entire user population:

```
14-million-entry list sprayed at 1 password per 30-minute round
→ 14 million rounds x 30 minutes = flatly infeasible, the attack would take longer than the useful lifetime of the credentials themselves

Realistic spray list: 5-20 CAREFULLY CHOSEN passwords, prioritizing:
- Seasonal/temporal patterns (CompanyName + current Season + Year,
  e.g. "Summer2026!", "Winter2025!")
- Default/common corporate patterns ("Welcome1", "ChangeMe123!",
  "Password1!")
- Org-specific terms from CeWL/OSINT scraping (per your Wordlists
  guide's Section 8-9), mangled with a single common suffix pattern
```

This is exactly the inverse tradeoff from your Wordlists guide's Section 1 point about targeted-beats-generic — here the constraint isn't compute cost (as it was for offline cracking against a slow hash), it's **wall-clock time enforced by the lockout window itself**, making list precision even more critical than in any offline scenario covered so far.

## 6. MFA as the Practical Ceiling on Spraying's Value

Worth stating plainly before Part 2's tooling details: a successfully sprayed valid credential against an account protected by properly-enforced multi-factor authentication does **not** grant access on its own — it grants a valid first factor, which may still have real value (credential reuse confirmation, a foothold for a subsequent MFA-fatigue/push-bombing attempt, or access to any secondary system that doesn't enforce MFA identically) but isn't automatic account takeover the way it would be against a single-factor target.

---

## PART 2: Practical Execution & Tooling

### 7. Kerbrute — Spraying Against Active Directory / Kerberos

Purpose-built for AD environments, exploiting a specific property of Kerberos pre-authentication: a failed Kerberos pre-auth attempt against a valid username returns a **different, distinguishable error** than one against an invalid username — meaning username validation (Section 3) and password spraying can both be performed via the same Kerberos-native mechanism, without touching the normal Windows login/lockout-counting path in exactly the same way an LDAP or SMB-based attempt would:

```
kerbrute userenum -d target.local usernames.txt
kerbrute passwordspray -d target.local usernames.txt 'Summer2026!'
```

userenum → validates which usernames exist, via Kerberos pre-auth error differentiation (Section 3's reconnaissance step, automated)  
passwordspray → sprays a single password across a username list, one round, respecting the interval discussed in Section 4 (run manually spaced apart, or scripted with a sleep/wait between invocations)

## 8. CrackMapExec / NetExec — Broader AD Spraying and Post-Spray Pivoting

A more general-purpose AD/SMB toolkit that includes spraying functionality alongside broader lateral-movement capability, useful when the goal extends past just validating a credential into using it immediately against multiple protocols:

```
netexec smb targets.txt -u usernames.txt -p 'Summer2026!' -
-continue-on-success
```

--continue-on-success → critical flag for spraying specifically - without it, the tool stops at the FIRST valid credential found, defeating the entire "check the whole population" point of a spray round

## 9. Spraying Cloud Identity Providers — MSOLSpray, o365spray, and Similar

Cloud identity endpoints (Microsoft 365/Azure AD, Okta, Google Workspace) each have their own dedicated spraying tools, since the authentication endpoint, error responses, and lockout/throttling behavior differ meaningfully from on-premises AD:

```
python3 MSOLSpray.py --userlist usernames.txt --password 'Summer2026!'
```

Cloud providers often implement SMART LOCKOUT or adaptive throttling rather than a simple fixed-threshold lockout - behavior that can differ per-IP, per-account-risk-score, or time-of-day, making Section 4's clean "wait W minutes" math meaningfully harder to calculate precisely and often requiring MORE conservative timing margins than an on-premises AD target with a known, static policy

## 10. OPSEC — Distributing Source IPs to Avoid Network-Level Detection

Beyond the account-lockout evasion Section 1 covers, a spray concentrated from a single source IP is trivially detectable/blockable at the network layer regardless of whether any individual account ever locks out:

- Rotating through multiple source IPs/proxies across spray rounds
- Respecting realistic request timing/spacing WITHIN a round, not just between rounds, to avoid triggering rate-based network detections
- Being mindful that MODERN SIEM/identity-protection tooling increasingly correlates "many failed logins across many DIFFERENT accounts from the same source" as its own distinct detection signature, specifically because password spraying has become common enough to have well-known detection patterns built for it
  - Section 1's lockout-evasion trick does NOT automatically evade THIS kind of aggregate behavioral detection

## 11. Testing / Engagement Methodology

1. Confirm authorization and SCOPE explicitly includes online authentication testing against live accounts - this carries real operational risk (potential lockouts affecting legitimate users) distinct from every offline technique earlier in this series, and needs explicit client sign-off on acceptable risk
2. Build the username list (Section 3) and validate a SAMPLE against a known-safe test account before running at scale
3. Determine the target's actual lockout policy (Section 4) - never assume a default; confirm it directly where possible
4. Select a small, high-probability password list (Section 5) - resist the urge to reuse a large offline-cracking wordlist here
5. Run ONE password per round, respecting the calculated safe interval, across the appropriate tooling for the target environment (Sections 7-9)
6. Document EVERY valid credential found, note whether MFA blocked further access (Section 6), and report account lockout policy gaps/weak-password prevalence as concrete, client-relevant findings

---

## 12. Legal & Ethical Note

Identical baseline authorization requirement to every prior guide in this series - but password spraying carries a MEANINGFULLY higher operational-risk profile than any offline technique covered so far, since it directly interacts with LIVE, real user accounts and can cause genuine business disruption (unintended lockouts, help-desk load, user confusion/alarm) even when technically "successful" and fully authorized. Explicit scope agreement on acceptable account impact, notification procedures, and timing windows (e.g. avoiding business hours) is standard practice and should be confirmed before any spray begins, not assumed.

## 13. Prevention — Detecting and Defending Against Spraying

- Enforce MFA universally (Section 6) - the single most effective mitigation, since it converts a successful spray from account takeover into merely a confirmed-weak-password data point
- Implement SMART/adaptive logout rather than naive per-account thresholds where possible (Section 9) - detecting the AGGREGATE "many accounts, one password, one source" pattern directly, rather than relying purely on Section 1's per-account counting, which spraying is specifically engineered to stay under
- Screen passwords against known-common/breached-password lists at set time (identical recommendation to your Hash Types and Wordlists guides' prevention sections) - directly removes the highest-probability candidates Section 2 relies on from the viable population entirely
- Monitor and alert on failed-login patterns aggregated ACROSS accounts and BY source IP/ASN, not just per-account failure counts - this is precisely the detection signature Section 10 describes attackers now actively trying to evade, meaning it remains a genuinely valuable, non-obsolete detection control
- Rate-limit or geofence authentication endpoints where organizationally reasonable, and monitor cloud identity provider's built-in risk/anomaly detection features (Azure AD Identity Protection and equivalents), which are frequently under-utilized despite being enabled by default in many tenants

---

## 14. Practical Lab Example

A local lab Active Directory environment (several TryHackMe/HTB rooms, or a self-built AD lab per common OSCP-prep guides) specifically built with a handful of weak-password accounts among many strong ones is the clearest way to see Section 2's "weakest link across a population" statistical principle proven directly, rather than theoretically.

Suggested practice order:

1. Set up a lab AD domain with a defined, KNOWN lockout policy and a mix of strong/weak-password test accounts
2. Enumerate valid usernames via Kerbrute (Section 7) before attempting any password guesses
3. Run a properly-spaced spray (Section 4's timing math) with a small, curated password list (Section 5) and confirm you recover the deliberately-weak account's credential WITHOUT triggering a single lockout event - the explicit proof of Section 1's core evasion mechanism

## Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

| Framework         | Mapping                                                                                                                                                                                                                                                                                                                                               |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CWE               | CWE-307 (Improper Restriction of Excessive Authentication Attempts - the lockout-policy gap this ENTIRE technique is built to route around, distinct from CWE-521/916 covered in earlier offline-cracking guides), CWE-521 (Weak Password Requirements, the root reason Section 2's statistical approach succeeds at all)                             |
| OWASP Top 10:2025 | A07:2025 Identification and Authentication Failures (the direct category for lockout-policy gaps and weak authentication controls, distinct from the Cryptographic Failures mapping used throughout your offline-cracking guides)                                                                                                                     |
| CVSS              | Typically High when successful against a non-MFA-protected account (AV:N, PR:N, low attack complexity), meaningfully reduced in practice when MFA is universally enforced per Section 6, even though the underlying credential weakness is unchanged                                                                                                  |
| Cyber Kill Chain  | Reconnaissance (Section 3's username enumeration) → Delivery/Exploitation (the spray attempts themselves, directly against the live authentication endpoint - notably, this collapses several kill-chain phases that offline cracking keeps separate, since exploitation happens in the same online interaction as the initial credential validation) |
| MITRE ATT&CK      | T1110.003 (Brute Force: Password Spraying - its own DEDICATED sub-technique, distinct from T1110.002's offline hash cracking used throughout your JtR/Hashcat/Rainbow Tables guides), T1589.002 (Gather Victim Identity Information: Email Addresses, covering Section 3's reconnaissance phase)                                                      |

## Quick Reference Cheat Sheet



CORE PRINCIPLE (inverted from every earlier guide in this series):

Offline cracking (JtR/Hashcat/Rainbow Tables): ONE account,  
MANY passwords, unconstrained by time - maximize depth

Password spraying: MANY accounts,  
FEW passwords, constrained by lockout window - maximize breadth

WORKFLOW:

1. Enumerate usernames (OSINT, Kerberos pre-auth, breach data)
2. Determine lockout policy (threshold N, window W)
3. Select 5-20 high-probability passwords (NOT a bulk wordlist)
4. Spray 1 password/round across ALL accounts, wait W+margin,  
repeat with NEXT password
5. Track hits; note MFA status on each success

TOOLING BY TARGET:

kerbrute userenum/passwordspray → on-prem AD / Kerberos  
netexec smb --continue-on-success → broader AD/SMB spraying  
MSOLSpray / o365spray → Microsoft 365 / Azure AD  
(equivalents exist per-provider for Okta, Google Workspace, etc.)

Root lesson: this is the ONLY technique in the series that attacks LIVE accounts directly rather than a dumped hash - every earlier guide's "rate-limit/lock out online attempts" prevention line was gesturing at THIS exact attack, and Section 1's per-account-vs-per-password inversion is precisely the design choice that makes a naive lockout policy insufficient against it on its own.